

MODULE 03

Copilot-Assisted C# Development

Day 1 · Core MVP Module



ADOBE STOCK PLACEHOLDER

Suggested search: "developer using an AI code assistant (wide hero banner)"

LEARNING OBJECTIVES

- Use prompt patterns for targeted refactoring and test generation.
- Review AI-generated code for quality and security.
- Improve maintainability of a legacy C# component.



ADOBE STOCK PLACEHOLDER

Suggested search: "GitHub Copilot code suggestion on screen (supporting image)"

WHY THIS MODULE MATTERS

This module converts theory into productivity by using Copilot against a controlled code surface.

The focus is safe acceleration, not blind generation.

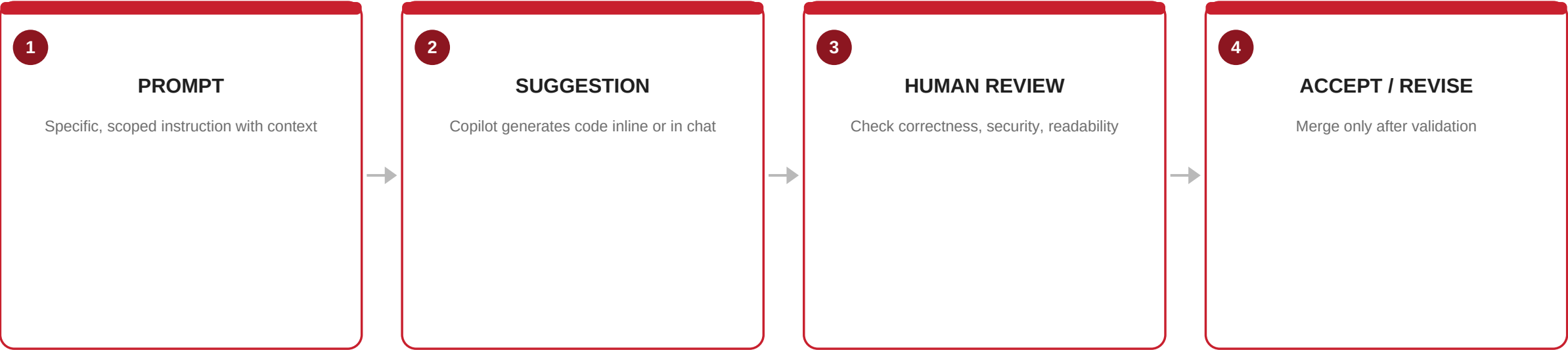


ADOBE STOCK PLACEHOLDER

Suggested search: "programmer pair programming with AI (concept image)"

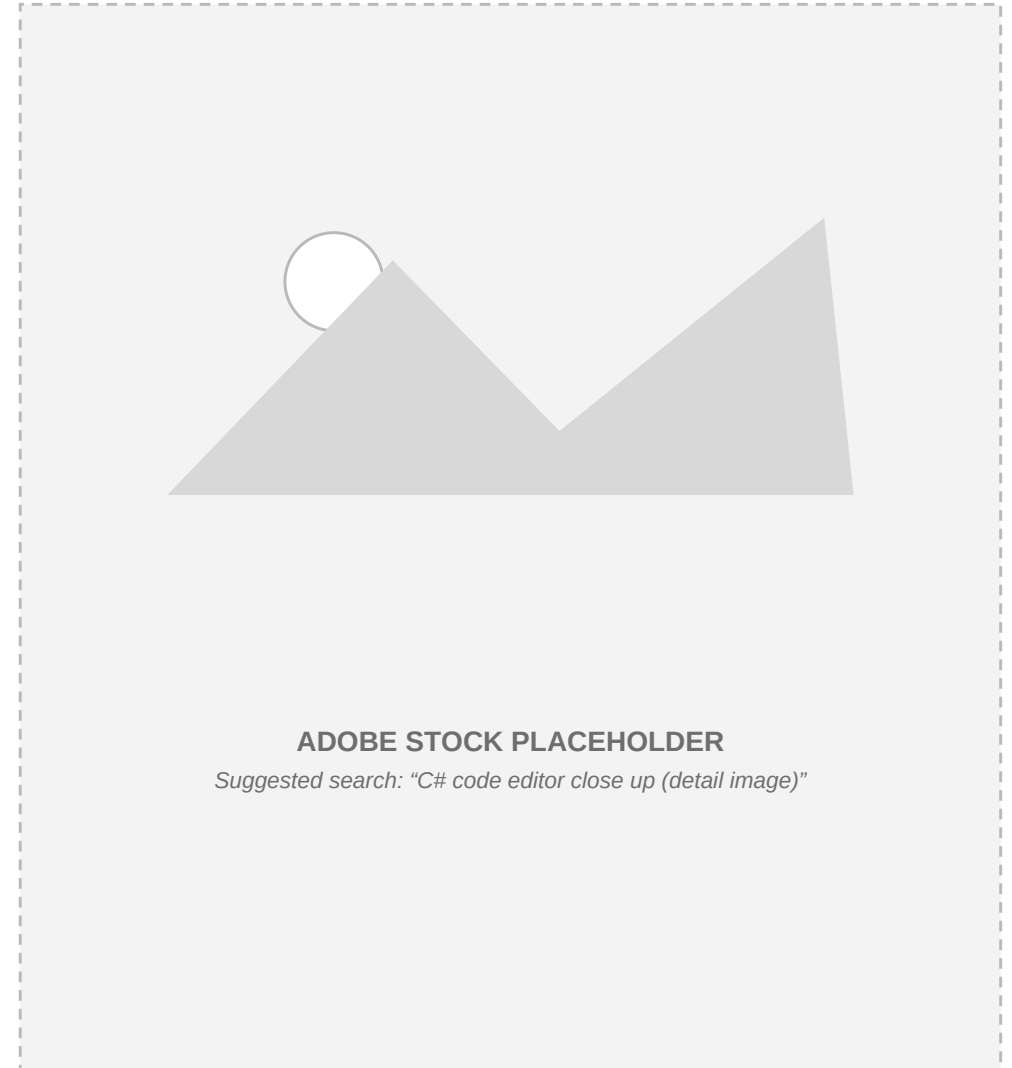
CONCEPT MAP

The safe-acceleration loop this module teaches — Copilot proposes, a human always validates.



FROM THE DOCS: GITHUB'S OWN BEST PRACTICES

- GitHub: break complex tasks into smaller, specific prompts for better suggestions.
- GitHub: be specific — provide example inputs, outputs, and implementations in the prompt.
- GitHub's own advice: ask Copilot Chat to explain a suggestion before you implement it.
- GitHub calls for manual review of AI code for functionality, security, readability, and maintainability.
- GitHub names writing tests and repetitive code as a core strength of Copilot suggestions.



COURSEWARE FOCUS

- Prompting patterns
- Refactoring legacy code
- Generating tests and documentation
- Governance and safe usage



ADOBE STOCK PLACEHOLDER

Suggested search: "developer using an AI code assistant (detail image)"

TOOLS & RESOURCES

Workshop Repo

microsoft/copilot-labs or microsoft/copilot-csharp-workshop

Practice prompting strategies, unit test generation (MSTest/xUnit), and refactoring.

Learning Path

Microsoft Learn: Develop with AI-powered code suggestions using GitHub Copilot

Source material for prompt engineering and governance best practices.

IDE Plugins

GitHub Copilot for Visual Studio / VS Code

Developer desktop integration for writing unit tests and XML documentation.

SUPPORTING ASSETS

- Code target: `course/repos/eShopOnWeb`
- Optional examples: `course/repos/samples`



ADOBE STOCK PLACEHOLDER

Suggested search: "GitHub Copilot code suggestion on screen (reference screenshot)"

LAB 03: COPILOT REFACTOR AND TESTS



ADOBE STOCK PLACEHOLDER

Suggested search: "programmer pair programming with AI (lab / hands-on photo)"

LAB STEPS

Goal: Refactor one legacy component and generate useful tests with Copilot.

Tools: VS Code or Visual Studio, .NET SDK, GitHub Copilot (license + sign-in required).

- Step 1 — Select one target class with known complexity issues.
- Step 2 — Prompt Copilot for a refactor plan.
- Step 3 — Apply the refactor incrementally.
- Step 4 — Generate unit tests for happy-path and failure-path behavior.
- Step 5 — Run the tests and review generated code for quality and security.
- Validation: the project compiles after refactor, and tests pass for the changed behavior.



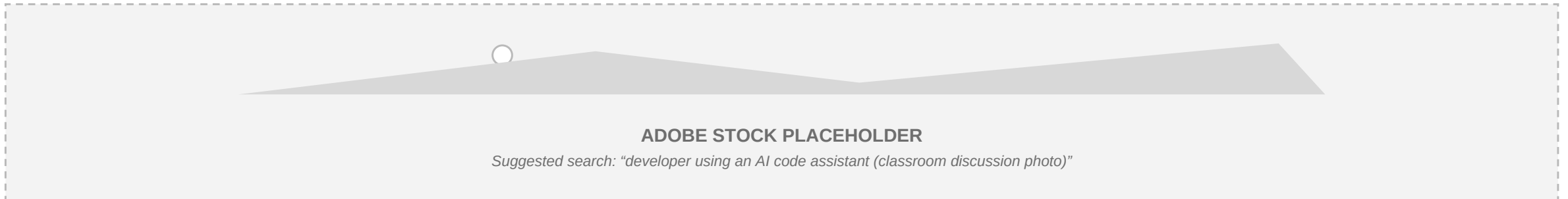
ADOBE STOCK PLACEHOLDER

Suggested search: "C# code editor close up (hands-on lab photo)"

KNOWLEDGE CHECK: Copilot-Assisted C# Development

What is the recommended practice after GitHub Copilot generates a refactor or test?

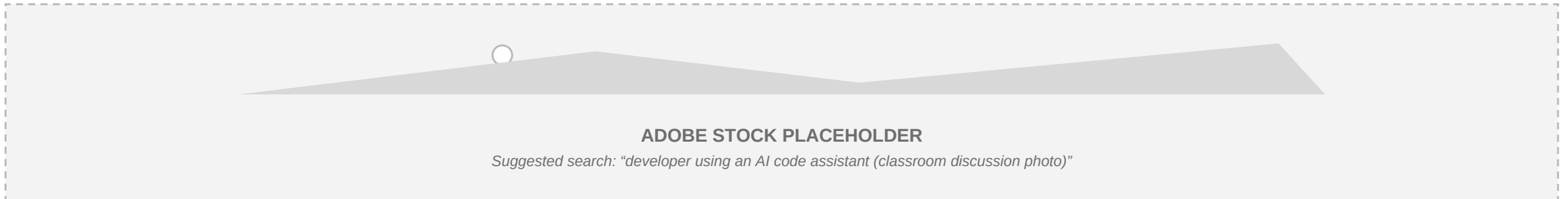
- A. Merge immediately to save time
- B. Review the output for correctness, security, and maintainability before accepting
- C. Disable code review since AI output is always correct
- D. Delete the original code without comparison



KNOWLEDGE CHECK: Copilot-Assisted C# Development

What is the recommended practice after GitHub Copilot generates a refactor or test?

- A. Merge immediately to save time
- B. Review the output for correctness, security, and maintainability before accepting**
- C. Disable code review since AI output is always correct
- D. Delete the original code without comparison



KNOWLEDGE CHECK: Copilot-Assisted C# Development

Per GitHub's own best practices, what should you do before implementing a Copilot suggestion you don't fully recognize?

- A. Merge it immediately
- B. Ask Copilot Chat to explain it and understand it first
- C. Disable all linting
- D. Delete your original code



KNOWLEDGE CHECK: Copilot-Assisted C# Development

Per GitHub's own best practices, what should you do before implementing a Copilot suggestion you don't fully recognize?

A. Merge it immediately

B. Ask Copilot Chat to explain it and understand it first

C. Disable all linting

D. Delete your original code



SOURCES & FURTHER READING

EXTERNAL DOCUMENTATION

■ Best practices for using GitHub Copilot

<https://docs.github.com/en/copilot/get-started/best-practices>

REFERENCED IN THIS REPOSITORY

■ `course/mvp-delivery/labs/lab-03-copilot-refactor-and-tests.md`

■ `course/repos/eShopOnWeb`

RECAP & SUCCESS CRITERIA

- Refactored code compiles, tests pass, and learner can explain the decisions made.
- Tier: Core MVP Module



ADOBE STOCK PLACEHOLDER

Suggested search: "programmer pair programming with AI (team success photo)"